COMPUTER ENGINEERING

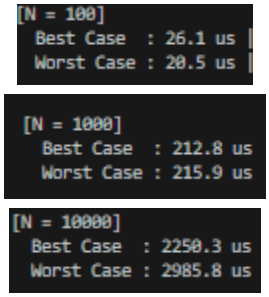
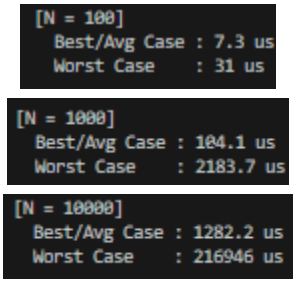
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 10/21/260 | **DATE SUBMITTED:** | 10/21/26 |
| **NAME:** | Estorco, Clint Hoven A.<br>Fernandez, Cedric S. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## ACTIVITY NO. 11 BASIC ALGORITHM ANALYSIS

### 1. PROCEDURE OUTPUT

| | Best case | Worst Case | Analysis |
|---|---|---|---|
| **Merge Sort** | O(n log n) | O(n log n) | The merge sort divides the array into two halves (log n) then follows the divide and conquer strategy making sure to compare at each lever of division and since its a divide and conquer strategy the best and worst case will always be (n log n) |
| **Quick Sort** | O(n log n) | $O\ (n^2)$ | First it selects the last element as a pivot then and when it comes to the best case it will be when the the pivot the array into exactly two halves making it only (n log n) after, but for the worst case when the pivot is divided into un even halves it will cause N recursive call leading to (n^2) |

| | Merge Sort | Quick Sort |
|---|---|---|
| **Best Case** | **N = 100**<br>Best Case : 26.1 us<br><br>**N = 1000**<br>Best Case : 212.8 us<br><br>**N = 10000**<br>Best Case : 2250.3 us | **N = 100**<br>Best Case : 7.3 us<br><br>**N = 1000**<br>Best Case : 104.1 us<br><br>**N = 10000**<br>Best Case : 1282.2 us |
| **Worst Case** | **N = 100**<br>Worst Case : 20.5 us<br><br>**N = 1000**<br>Worst Case : 215.9 us<br><br>**N = 10000**<br>Worst Case : 2985.8 us | **N = 100**<br>Worst Case : 31 us<br><br>**N = 1000**<br>Worst Case : 2183.7us<br><br>**N = 10000**<br>Worst Case : 216946 us |
| **Screen Shot** |  |  |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**Linear search**

```
Algorithm linearSearchLL(head, dataFind):
compare = 0
nodes = head
while nodes != nullptr do
    compare = compare + 1
    if nodes.data == dataFind then
        return compare
    nodes = nodes.next
 return compare
```

**Pseudo code analysis:**

- Linear search's worst case time is O(n); that is, it takes time linearly proportional to the size of the data set. This one-to-one chance is reflected in the execution times in the following empirical data: When N = 1,000, the execution time is 0.011 milliseconds, while for N = 100,000 the execution time is 0.806 milliseconds. Also, it has constant space complexity of O(1) which makes it light on memory consumption but not

effective in searching large data sets.

**Binary search**

```
Algorithm binarySearchArr(data, arrSize, dataFind):
 low = 0
 high = arrSize - 1
  while low <= high do
     middle = integer((low + high) / 2)
     if dataFind == data[middle] then
        print "the data was found"
        return
     else if dataFind < data[middle] then
        high = middle - 1
    else
       low = middle + 1
 print "the data was not found"
```

**Pseudo code analysis**

- Binary search is an efficient method, with a worst-case time of O(log n), since it halves the search space each time. For all test set input sizes up to N = 100,000 the worst case execution time is nearly instant: 0.0006 ms. When you have a large, pre-sorted dataset, binary search can be by far the better option due to its constant O(1) space complexity.

## Space and time complexities

| Algorithm | Best case time complexity | Worst case time complexity | Space complexity |
|---|---|---|---|
| **Liniar search** | O(1) | O(n) | O(1) |
| **Binary search** | O(1) | O(log n) | O(1) |

| Input Size (N) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| **1,000** | 0.0004 ms | 0.011 ms |
| **10,000** | 0.0002 ms | 0.0765 ms |
| **100,000** | 0.0006 ms | 0.806 ms |
| **Screen shot** | N = 1,000<br>`Binary Search (Worst-Case): 0.0004 ms`<br>N = 10,000<br>`Binary Search (Worst-Case): 0.0002 ms`<br>N = 100,000<br>`Binary Search (Worst-Case): 0.0006 ms` | N = 1,000<br>`Linear Search (Worst-Case): 0.011 ms`<br>N = 10,000<br>`Linear Search (Worst-Case): 0.0765 ms`<br>N = 100,000 |

| | | |
|---|---|---|
| | | Linear Search (Worst-Case): 0.806 ms |

| Analysis |
|---|
| The document here contains the pseudocode for both linear search and binary search, theoretical evaluations of each, and empirical benchmarks over input sizes of N = up to 100,000 characters per size. The implication of linear search's worst-case time complexity is pronounced as the execution times are linearly increasing as the number of elements in the data set grows up to 0.806 ms. On the other hand, binary search remains with logarithmic running time O(log n) and constant auxiliary space O(1) and performs in 0.0006 Ms for N = 100,000 (fraction of a millisecond) for huge data sets.. |

| Conclusion |
|---|
| **Estorco** |
| This activity emphasizes the effects of algorithm design on the efficiency of algorithm execution when the data size becomes larger. Theoretical and experimental analysis enabled us to make predictions regarding Merge Sort that has consistent time complexity of $O(N \log N)$ irrespective of the input, and that Quick Sort has $O(N \log N)$ best/average case complexity, but has $O(N^2)$ worst case complexity. Analogously, an operation of search conducted on benchmarks indicates that Binary Search performs almost instantaneous $O(\log N)$ lookups on a predetermined sorted data whereas Linear Search performs linear $O(N)$ lookups. In general the most important factors to consider when selecting the best algorithm are related to data preparation, data structure in memory and the frequency of search operations. |
| **Fernandez** |
| In this activity it gave me a deeper understanding on have the types of algo can dictate how efficiently a compiler can run and search or travel through the data like when comparing linear search linked list and binary search on sorted array which made me understand that linked list uses sequential pointer traversal, whereas contiguous arrays allow binary search to eliminate half of the remaining search space with every comparison to achieve O(log N) efficiency. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction, code assistance, explanation of concept**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **explanation of concept, and trouble shooting**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.